



Integer-based accurate conversion between RGB and HSV color spaces[☆]



Vladimir Chernov^{*}, Jarmo Alander, Vladimir Bochko

Department of Electrical Engineering and Energy Technology, University of Vaasa, Finland

ARTICLE INFO

Article history:

Received 2 December 2014

Received in revised form 5 August 2015

Accepted 6 August 2015

Available online 19 August 2015

Keywords:

HSV

RGB

Integer algorithm

Color space conversion

ABSTRACT

This paper introduces a new fast integer-based algorithm to convert the RGB color representation to HSV and vice versa. The proposed algorithm is as accurate as the classical real-valued one. The use of only integer operations increases performance and portability. Performance measurement results show a speed gain of about two times when compared with the classical C++ language implementation on PC platforms. Lookup tables are not involved, thus the memory usage is minimal. The resulting HSV color can be packed into 48 bits. The proposed method can safely replace the commonly used floating-point implementation.

© 2015 Elsevier Ltd. All rights reserved.

1. Introduction

The Hue Saturation Value (HSV) color space was created by Alvy Ray Smith in 1978 [1]. Initially, it was designed to improve a color picking interface in computer graphics software. The HSV model is a popular tool for color mixing since it is well consistent with human color perception. Furthermore, the HSV color space is often used in image processing and computer vision fields.

HSV space is well proven in image segmentation. Ganesan et al. used HSV color histogram for segmentation of region of interest in satellite images [2]. Liu et al. developed a method of fog level detection in highspeed road traffic based on image HSV color histogram [3]. Feng et al. proposed online method for rare colored capsule detection based on RGB and HSV color space [4].

Several works apply HSV for skin detection. Rahman et al. combined HSV and YCbCr color spaces in a new human skin detection method [5]. Bhowmick et al. applied HSV and YCbCr mixed color space for skin color segmentation in hand gesture recognition method for English alphabets [6]. Chaudhary and Gupta used HSV in dynamic color space switching system for skin pixel segmentation [7].

HSV components are often used as feature vectors for image retrieval and classification. Rashno et al. proposed a feature extraction schema that includes color features in RGB and HSV domain [8]. Raja et al. used colors from HSV model as low level features in robust indoor/outdoor scene classifier [9]. Archana et al. applied HSV in multiple face detection method with accuracy of 93% [10].

Fast RGB to HSV conversion algorithm is required for video processing. Wang and Zhao developed robust image chroma-keying method using HSV colors [11]. Chernov et al. proposed underwater video enhancement method based on HSV color space [12]. Guo et al. adjusted S and V channels in order to improve visibility and fidelity of underwater images [13].

[☆] Reviews processed and recommended for publication to the Editor-in-Chief by Area Editor Dr. E. Cabal-Yepez.

^{*} Corresponding author. Tel.: +358 40 6647727.

E-mail addresses: vladimir.chernov@student.uva.fi (V. Chernov), jal@uva.fi (J. Alander), vbochko@gmail.com (V. Bochko).

The use of HSV rather than RGB space is beneficial in texture analysis. Paschos [14] demonstrates that perceptually uniform spaces, such as HSV and $L^*a^*b^*$, outperform nonuniform RGB. He concludes, according to experimental results, that HSV could be a superior color space, compared to RGB, for color texture analysis. In noisy conditions the HSV performs better than the $L^*a^*b^*$.

The HSV model is designed as a transform of the RGB space. Practically, the only way to obtain the color in HSV is to convert it from RGB first. Therefore, characteristics of the H , S , and V components are dependent on the transformation algorithm. The classical conversion formulas, proposed by Smith [1], assume computations on real (not integer) numbers. Otherwise, the result would not have the expected precision. This causes the consideration for the H , S , V to be real numbers as well. The conversions have a classical implementation in programming languages with floating-point variables involved. As a reference of such implementation, the source code [15] from the GIMP editor (GNU Image Manipulation Program) is recommended by the authors.

In most cases, the classical floating-point realization is sufficient. However, there is a branch of algorithms that aims to improve the performance and portability of the color space conversions. A reasonable way in such improvement is to use integer (fixed-point) arithmetic and data types. Note, that the terms fixed-point and integer are practically identical in this context and used as synonyms in this paper.

Kabi et al. switched from floating-point to fixed-point arithmetic in order to reduce power consumption, time and market price of handheld devices for the application of speech-based emotion recognition [16]. Dobashi et al. proposed a fixed-point tone mapping operation (TMO) as a replacement for a floating-point TMO in order to reduce a computational cost for generating a low dynamic range (LDR) image from a high dynamic range (HDR) image [17].

The main practical reasons motivating the usage of the integer instead of floating-point arithmetic are performance and portability. For most processors, integer arithmetic is faster and well-suited to optimize hardware implementations like FPGA and allow for nice embedded system realizations. In addition, the integer-based algorithm is able to be run on hardware without an FPU (Floating-Point Unit or math coprocessor).

The integer data types for both RGB and HSV color schemes are preferable in many applications. The integer-based representation is generally more robust. Following good programming practices, the float and double types should not be used where exact results are required. This rule is applicable for many pixel manipulation applications, where floating-point representation is redundant. However, the straightforward change to integer types in the classical implementation is impossible without precision loss.

Liu et al. [18] proposed a fast algorithm for color space conversion based on fixed-point calculations on digital signal processors (DSPs). In the publication, the following conversion methods are included: YCbCr (luminance, chrominance-blue, and chrominance-red) to RGB, RGB to HSV, and YCbCr to HSV. The authors replaced floating-point multiplications with fixed-point shifts, 16-bit fixed-point multiplications, and additions. They used supporting small-sized (225 bytes and 256 bytes) lookup tables (LUT) in order to compensate for rounding error and to realize the 16-bit fixed-point multiplication. According to the experimental results, the authors achieved the error in acceptable range, maximum equal to 3. At the same time, they significantly improved computational speed, about 10 times faster on a fixed-point DSP and about 1.41 times faster on a personal computer (PC).

This paper introduces an integer-based algorithm to convert color representation from RGB to HSV and vice versa. The proposed algorithm is as accurate as the classical one. Each color from the RGB space has a unique equivalent in the HSV space. All HSV and RGB components in the proposed conversions are integer numbers. The use of floating-point variables is excluded entirely. The implementation of the proposed algorithm is fast, which is important for real-time image processing applications. Performance measurement results show a speed gain of about two times when compared with the classical implementation [15] on PC platform. Lookup tables are not involved. Therefore, the memory usage is minimal. The proposed algorithm can safely replace the classical one. If the floating-point data types are still needed for complex math operations, the HSV channels can be type-casted afterward.

This paper is organized as follows. In Sections 2 and 3 the theoretical background about RGB and HSV models, respectively, is given. In Section 4 the classical real-valued conversion algorithm is explained. The proposed integer-based algorithm is presented in Section 5. The performance analysis is done in Section 6. The experimental results are shown in Section 7. Finally, the paper is concluded in Section 8.

2. The RGB model

The RGB model encodes a color using three components: red (R), green (G), and blue (B). It is an additive color scheme that employs the principle of human eye functionality, i.e., sensitivity of three types of cones in retina to specific light spectra [19]. Thus visible colors can be reproduced by adding various intensities of red, green, and blue lights. The concept of RGB mixing is widely utilized in display and camera devices that makes this color model essential for most computer graphics applications.

RGB space can be visualized in the form of a cube in a three-dimensional Cartesian coordinate system (Fig. 1). The black color is in the axis origin while the white color is located in the diagonally opposite corner. The rest of the vertices represent primary colors (red, green, and blue) and secondary colors (cyan, yellow, and magenta).

The standard 24-bit RGB format is required for the proposed integer algorithm. In the 24-bit RGB pixel, all components have 8-bit depth. The total number of possible colors in this case is $2^8 \cdot 2^8 \cdot 2^8 = 16,777,216$. The components are interpreted as unsigned integers in range $[0, 255]$, which is exactly the value range of a single byte.

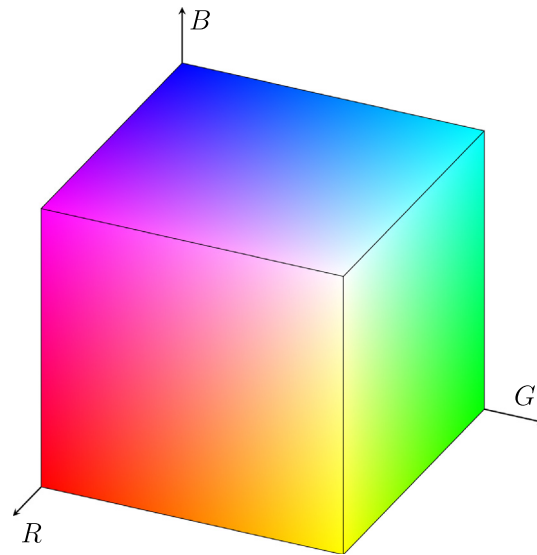


Fig. 1. RGB color cube. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Typically, the pixels of raw images are given in this format. However, in the classical conversion algorithm, RGB channels are required to be floating-point numbers in range $[0, 1]$, which causes the type-casting and division operations. In the proposed algorithm, the channels are processed as in the basic 24-bit format. The order of the components is not important since both algorithms process them independently.

The RGB scheme has a number of disadvantages. It is not intuitive for human interpretation. The luminance and chrominance properties are not separated. The Euclidean distance cannot be applied for correctly capturing color differences [20]. All the components are highly correlated with each other and in addition it is highly sensitive to illumination changes and noise.

3. The HSV model

The HSV model defines color in terms of three components: hue (H), saturation (S), and value (V). It is an approximately uniform perceptual color space [14] that correlates well with human color sensation. In uniform spaces, the measured color difference (Euclidean distance) is proportional to human perception of such difference. In comparison, the RGB is not uniform.

The hue component (H) represents a true color, e.g., red, yellow, green, cyan, blue, magenta, etc. Its pure values contain primary colors, secondary colors, and linear mixtures between adjacent pairs of them. As a physical measure, the hue is related to the wavelength of light.

The saturation component (S) represents color purity. It gives a measure of how much the true color is diluted by white. Low percentage of the S channel produces grayish color while high percentage produces deep color. Decreasing S (desaturation) corresponds to increasing whiteness.

The value component (V) is an analog of brightness. It measures the departure of the true color from black i.e. zero energy. Decreasing V (devaluation) corresponds to increasing blackness.

HSV color space was designed as a deformation of the RGB color cube. In other words, the HSV is a different coordinate system describing the RGB model (Fig. 2). The HSV is geometrically represented as a hexagonal cone (hexcone). The term hexcone was used by Smith in his original paper [1].

The central vertical axis (the gray axis) of the hexcone corresponds to the main diagonal of the color cube. It contains achromatic colors (grays). Value (V) varies along the gray axis from black at minimum to white at maximum. At least one of R , G , or B equals V , and none is larger.

The horizontal cross-sections of the hexcone are hexagons of different sizes. As value (V) changes, the sizes of the hexagons are also changed. Each one is larger than the preceding one when moving from black to white. The cross-section that corresponds to black color degrades to a point.

The Hue (H) and saturation (S) pair specifies a point in the hexagon. The hue (H) is a circular quantity. It is visualized by the angle around the gray axis. According to the classical layout [1], 0° is red, 60° is yellow, 120° is green, 180° is cyan, 240° is blue, and 300° is magenta. The saturation (S) is the distance from the gray axis along the horizontal axis. It is a relative quantity. The maximum S corresponds to the longest possible distance at the given H . A point with the maximum S lies on the boundary of the hexagon. Hue (H) is undefined and saturation (S) is equal to 0 along the gray axis.

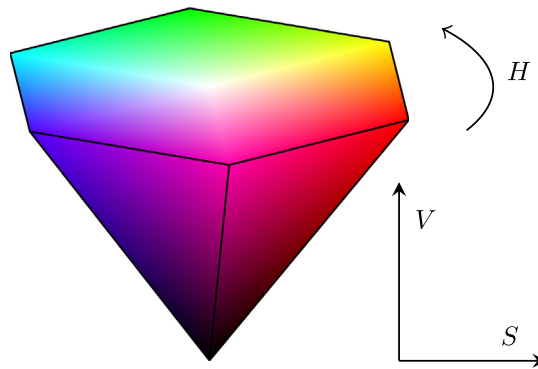


Fig. 2. HSV hexcone.

Each H and S pair belongs to one of the six sectors of the hexagon. The sector can be defined by H (as angle), or by dominance of R , G , and B components. In Table 1 the properties of all six sectors are presented. The sector number (I) is important for the conversions. H is derived as its fraction plus the sector offset. The hue fraction (F) is the partial value of H inside the sector. The sector offset is the integer amount of edges (counted counter-clockwise) multiplied by the edge length (E). Therefore, H is computed as $H = (E \cdot I + F)/r$, where r is a scaling factor for adjusting the range.

The ranges of the components vary in different realizations. Since hue (H) is angular, it is often expressed in degrees $[0, 360]$. Another convenient representation, that is suitable for all components, is percentage (from 0% to 100%). In Smith's work [1], as well as in the classical implementation [15], each of the components is in the range $[0, 1]$. In this paper, when the real-valued algorithm is considered, the H , S , and V are also assumed to be in the interval $[0, 1]$. Such assumption, simplifies the hue formula to $H = (I + F)/6$, for $E = 1$ and $r = 6$. A constant scaling factor in different realizations can be ignored without loss of generality.

4. Classical real-valued conversion algorithm

Smith in his paper [1] presented transformation algorithms for forward and backward conversions between the RGB and HSV models.

The forward conversion (from RGB to HSV) requires the following steps:

1. Find maximum (M) and minimum (m) of R , G , and B .

$$M = \max(R, G, B)$$

$$m = \min(R, G, B)$$

2. Assign $V = M$.

3. Calculate delta (d) between M and m .

$$d = M - m$$

4. If d is equal to 0 then assign S with 0 and return. H is undefined in this case.

5. Calculate S as a ratio of d and M .

$$S = \frac{d}{M}$$

6. Calculate H depending on what are M and m .

Table 1
Hue sectors.

Index (I)	Hue range in degrees	Order of R , G , and B
0	$[0, 60)$	$R > G > B$
1	$[60, 120)$	$G > R > B$
2	$[120, 180)$	$G \geq B \geq R$
3	$[180, 240)$	$B > G > R$
4	$[240, 300)$	$B \geq R \geq G$
5	$[300, 360)$	$R \geq B \geq G$

$$H = \frac{1}{6} \cdot \begin{cases} \frac{G-B}{d}, & \text{if } M = R \text{ and } m = B \\ 1 + \frac{R-B}{d}, & \text{if } M = G \text{ and } m = B \\ 2 + \frac{B-R}{d}, & \text{if } M = G \text{ and } m = R \\ 3 + \frac{G-R}{d}, & \text{if } M = B \text{ and } m = R \\ 4 + \frac{R-G}{d}, & \text{if } M = B \text{ and } m = G \\ 5 + \frac{B-G}{d}, & \text{if } M = R \text{ and } m = G \end{cases}$$

The conversion involves ordering of R , G , and B components, i.e. finding the minimum, and maximum of them. If they are equal, then it is the achromatic case when saturation (S) is 0 and hue (H) is undefined. Value (V) is always equal to the maximal RGB component. The formula for saturation is $S = \frac{\max(R,G,B) - \min(R,G,B)}{\max(R,G,B)}$. The formula for hue (H) includes computation of its fraction part and summation with appropriate sector offset. The fraction of hue is computed as $F = \frac{\text{mid}(R,G,B) - \min(R,G,B)}{\max(R,G,B) - \min(R,G,B)}$. The middle component and sector offset are determined according to previously found maximum (M) and minimum (m). Finally, H is divided by 6 in order to change the range to $[0, 1]$.

The backward conversion (from HSV to RGB) can be done by the following steps:

1. If S is equal to 0 then assign all R , G , and B with V and return.
2. Multiply H by 6.

$$H = 6 \cdot H$$

3. Determine the hue sector index (I) by taking the largest previous integer (floor) of H .

$$I = \lfloor H \rfloor = \text{floor}(H)$$

4. Determine the hue fraction (F) by subtracting I from H .

$$F = H - I$$

5. Calculate values (M , N , and K) of RGB components.

$$M = V \cdot (1 - S)$$

$$N = V \cdot (1 - S \cdot F)$$

$$K = V \cdot (1 - S \cdot (1 - F))$$

6. Assign R , G , and B with appropriate values M , N , and K , ordered according to I .

$$[R \quad G \quad B] = \begin{cases} [V \quad K \quad M], & \text{if } I = 0 \\ [N \quad V \quad M], & \text{if } I = 1 \\ [M \quad V \quad K], & \text{if } I = 2 \\ [M \quad N \quad V], & \text{if } I = 3 \\ [K \quad M \quad V], & \text{if } I = 4 \\ [V \quad M \quad N], & \text{if } I = 5 \end{cases}$$

Both the forward and backward conversions are realized in the well-known implementation [15].

5. Proposed integer-based conversion algorithm

In this section the proposed algorithm is explained in detail. First, the overview of the problem is given, and then the solution is presented.

The most challenging part of the integer-based algorithm is the computation of S and H while keeping the possibility to recover the original R , G , and B . The S component, in the classical conversion, is computed as a ratio of delta (d) and V . The real-valued formula for H contains fractional part $F = (\text{mid}(R,G,B) - \min(R,G,B))/d$. Both formulas require one division operation.

The well-known technique to increase the accuracy of the integer division is to shift the dividend left by a certain number of bits. This improves accuracy but does not guarantee lossless backward conversion. Generalization of the problem gives a fraction $x = \lfloor a/b \rfloor$ and recovering of numerator $y = x \cdot b$, where all numbers are integers. In the integer division the remainder is discarded, therefore in general $y \neq a$.

The analysis of ranges for all variables gives a clue to make a proper division and reconstruction formula. The range of d is $[1, 255]$. The special case $d = 0$ is handled separately (step 4 of the classical forward conversion, see Section 4). V is equal to $\max(R,G,B)$, therefore $d \leq V \leq 255$. Hence, assuming reduced fractions, there are 19,820 unique values of $S = d/V$

(the number was computed numerically). This is a theoretical count of all meaningful values of S that can be obtained from 24-bit RGB.

In order to perform accurate integer-based conversions related to S , the following pair of equations is proposed:

$$S = \frac{d \cdot 2^{16} - 1}{V} = \frac{(d \ll 16) - 1}{V} \quad (1)$$

$$d = \frac{S \cdot V}{2^{16}} + 1 = ((S \cdot V) \gg 16) + 1 \quad (2)$$

All the coefficients were found experimentally by the authors. For arbitrary d and V from given ranges, Eq. (1) produces equivalent results for integer values as the classical formulas for real values. One criteria for correctness is the amount of unique values. Eq. (1) produces 19,820 unique saturations (S) which is equal to the nominal evaluation (claimed earlier). Furthermore, Eq. (2) always recovers original d from given S and V . This is essential for lossless backward conversion.

According to Eq. (1), the smallest value of S is 257 (for $d = 1$ and $V = 255$). The largest value of S is 65,535 (for $d = 1$ and $V = 1$), which is perfectly fit to 16 bits (2 bytes).

The logic behind the computation of the fractional part of H is similar to that for saturation (S). The amount of unique values of $F = (c - m)/d$, where $c = \text{mid}(R, G, B)$ and $m = \min(R, G, B)$, is 19,821. The following equations related to F are proposed:

$$F = \frac{(c - m) \ll 16}{d} + 1 \quad (3)$$

$$c = ((F \cdot d) \gg 16) + m \quad (4)$$

Eq. (3) produces the correct amount of unique F for the given ranges. The original c is recovered without errors by Eq. (4).

The smallest integer-based value of F is 1 (for $c = m$ and $d = 1$). The largest value of F is 65,537 (for $c - m = 1$ and $d = 1$). The hexagon edge length (E) is equal to largest F ($E = 65,537$). Therefore, the maximum value of an integer-based H is $6 \cdot E = 393,222$ which requires 19 bits. Since one byte is a minimum addressable piece of data in the system, and there is usually no 24-bit data type in programming languages, it is reasonable to store H in a 32-bit integer variable. For half of the hexagon sectors ($I = 1, 3, 5$), F must be inversed ($F = E - F$) in order to fit the classical color layout.

Computation of V is straightforward. Since it is equal to the maximum of R , G , and B , the type can be changed to 8-bit integer without additional changes of the classical algorithm.

In order to summarize this proposal, the following pseudo code listing represent the integer-based forward conversion algorithm (from RGB to HSV):

1. Find the maximum (M), minimum (m) and middle (c) of R , G , and B .

$$M = \max(R, G, B)$$

$$m = \min(R, G, B)$$

$$c = \text{mid}(R, G, B)$$

2. Assign V with M .

$$V = M$$

3. Calculate the delta (d) between M and m .

$$d = M - m$$

4. If d is equal to 0 then assign S with 0 and return. H is undefined in this case.

5. Determine sector index (I).

$$I = \begin{cases} 0, & \text{if } M = R \text{ and } m = B \\ 1, & \text{if } M = G \text{ and } m = B \\ 2, & \text{if } M = G \text{ and } m = R \\ 3, & \text{if } M = B \text{ and } m = R \\ 4, & \text{if } M = B \text{ and } m = G \\ 5, & \text{if } M = R \text{ and } m = G \end{cases}$$

6. Calculate integer-based saturation (S).

$$S = \frac{(d \ll 16) - 1}{V}$$

7. Calculate the fractional part of hue (F).

$$F = \frac{(c - m) \ll 16}{d} + 1$$

8. Inverse F for specific sectors (1, 3, and 5), by subtracting F from the edge length constant (E).

if $I = 1$ or $I = 3$ or $I = 5$ then $F = E - F$

9. Calculate the final integer-based hue (H).

$$H = E \cdot I + F$$

The pixel depth of resulting HSV color is 48 bits. V is stored in 8 bits (1 byte), S in 16 bits (2 bytes), and for the H component at least 19 bits are required, therefore in most systems 24 bits (3 bytes) must be reserved for H .

The listing for the proposed backward conversion algorithm (from HSV to RGB) is the following:

1. If S or V is equal to 0 then assign all R , G , and B with V and return.
2. Calculate delta (d).

$$d = ((S \cdot V) \gg 16) + 1$$

3. Calculate m , the minimal value of R , G , and B .

$$m = V - d$$

4. Determine the hue sector index (I) using the hexagon edge length constant (E).

$$I = \begin{cases} 0, & \text{if } H < E \\ 1, & \text{if } H \geq E \text{ and } H < 2E \\ 2, & \text{if } H \geq 2E \text{ and } H < 3E \\ 3, & \text{if } H \geq 3E \text{ and } H < 4E \\ 4, & \text{if } H \geq 4E \text{ and } H < 5E \\ 5, & \text{if } H \geq 5E \end{cases}$$

5. Calculate the fractional part of hue (F).

$$F = H - E \cdot I$$

6. Calculate the middle component (c).

$$c = ((F \cdot d) \gg 16) + m$$

7. Assign R , G , and B according to the sector index (I).

$$[R \quad G \quad B] = \begin{cases} [V \quad c \quad m], & \text{if } I = 0 \\ [c \quad V \quad m], & \text{if } I = 1 \\ [m \quad V \quad c], & \text{if } I = 2 \\ [m \quad c \quad V], & \text{if } I = 3 \\ [c \quad m \quad V], & \text{if } I = 4 \\ [V \quad m \quad c], & \text{if } I = 5 \end{cases}$$

Note, that steps 1 and 5 in the forward algorithm can be merged for the sake of optimization, as well as steps 4 and 7 in the backward case. See a C++ implementation [21] as an example.

6. Performance analysis

In this section a comparative performance analysis of both algorithms is presented. The performance is estimated by counting the basic operations and theoretical CPU clock cycles they take to execute. The C++ implementations of the integer-based [21] and classical [15] algorithms are analyzed.

In general, it is difficult to make a realistic estimation of algorithm performance when taking into account all varieties of platforms. Results may vary a lot if the algorithm is running on embedded system, FPGA, or desktop computer. However, it is possible to make rough assumptions according to optimization guide for modern (x86 family) CPUs [22]. Such rough analysis is presented in this section. It is not claimed to be full. There are many uncertain factors in CPU architecture like instruction pipelining, branch prediction (if-else statement) and compiler optimizations.

The key factor for this analysis is the presence and characteristics of floating-point unit (FPU) in hardware. If floating-point operations are slow, then the use of the integer-based algorithm is obviously more beneficial. In modern processors the operations with floating-point variables are almost as fast as with integers. But, even on modern hardware, the integer operations are preferable in many cases.

Table 2

Number of operations and memory usage.

Operation	Integer-based forward [21]	Classical forward [15]	Integer-based backward [21]	Classical backward [15]
Bitwise shifts	2	0	2	0
Additions, subtractions	6	3	5	5
Multiplications	1	1	3	5
Divisions	2	2	0	0
Comparisons	4	7	4	6
Memory [bytes]	22	36	15	34

The number of operations for all algorithms is presented in Table 2. The last row shows the memory usage, i.e. how many bytes are required to execute certain algorithm for one color pixel. The proposed algorithm requires less memory than the classical one. 14 and 19 bytes less for forward and backward conversions respectively.

All counting is done for the worst scenario, i.e. non-achromatic colors, deepest if-else nesting. Plus and minus operators are assumed to be equivalent.

For the floating-point variables, the theoretical operation costs [22, p. 33] are the following: addition takes 3–6 clock cycles, multiplication 4–8 clock cycles, and division 14–45 clock cycles. Floating-point comparisons are inefficient when the floating-point stack registers are used.

When dealing with the 24-bit RGB colors, the software that uses the real-valued algorithm cannot avoid performing lots of type castings between integer and float types. This can be very time consuming when taking into account the number of pixels and color components. Integer to float conversion takes 4–15 clock cycles [22, p. 41]. Typically, the conversion from float to integer takes no less than 50–100 clock cycles [22, p. 42]. The reason is that the C/C++ standard specifies truncation so the floating-point rounding mode has to be changed to truncation and back again.

According to the optimization guide [22, p. 31], simple integer operations such as addition, subtraction, comparison, bit operations and shift operations are very fast. They take only one clock cycle on most microprocessors. Integer multiplication takes 3–4 clock cycles.

The bottleneck of the proposed algorithm is the integer division operation which is very slow. It can take as many as 40–80 clock cycles [22, p. 31]. Other operations are relatively cheap and their amount does not differ much from the classical algorithm.

The integer-based backward conversion is obviously faster than the floating-point based. It performs fewer multiplications. Integer multiplication is cheaper than the float one. In addition, the classical backward algorithm implementation contains the expensive float to int cast.

Summarizing all the listed facts, the use of integer-based implementation may be theoretically beneficial even on x86 family processors. Slow integer divisions are compensated by eliminating the need of conversions between integer and float variables. Other operations, like comparisons and multiplications are faster to do with integers as well. These estimations are verified by practical tests in the next section.

7. Experimental results

In this section the practical aspects for usage of the proposed algorithm are discussed. The testing method is explained, and the correctness of results is shown together with performance and speed characteristics.

In order to test the correctness of the proposed algorithm, the following method was used: for each color from the 24-bit RGB cube. A color was converted forward to HSV and back to RGB using two methods: the classical real-valued and the proposed algorithms. The error between the original and expected results was measured. The results for both methods were compared, and statistics was saved.

The proposed algorithm perfectly passes all the tests: the RGB colors obtained from the backward conversion are always equal to the original ones. There are exactly 16,777,216 unique HSVs that is equal to the amount of all possible 24-bit RGBs ($2^8 \cdot 2^8 \cdot 2^8$). The statistical results for values of separate components are also equal to the ones theoretically predicted (see Section 5).

An additional experiment was to compare HSV colors of the proposed algorithm with those obtained by the classical algorithm. In order to convert an integer-based color component into a floating-point representation, it must be divided by the maximum of this component. After that, all the values are in range [0, 1]. The result of this experiment showed that the obtained integer-based HSV colors can be safely converted into classical real-valued HSVs. Comparisons are equal for all colors within small error marginal $\epsilon = 0.00002$.

There is no impact in the color conversion process that can be detected visually. It can be seen by observing color palettes. Generated by both algorithms palettes look identical (Fig. 3).

The speed characteristics were tested on PC platform (Intel Core i5 2.67 GHz CPU) with Ubuntu 14.04.1 OS (Linux 3.13.0-32-generic kernel) and GCC 4.8.1 compiler. The two versions: debug (non-optimized) and release (optimized) were compiled (-O0 and -O3 compiler flags respectively).

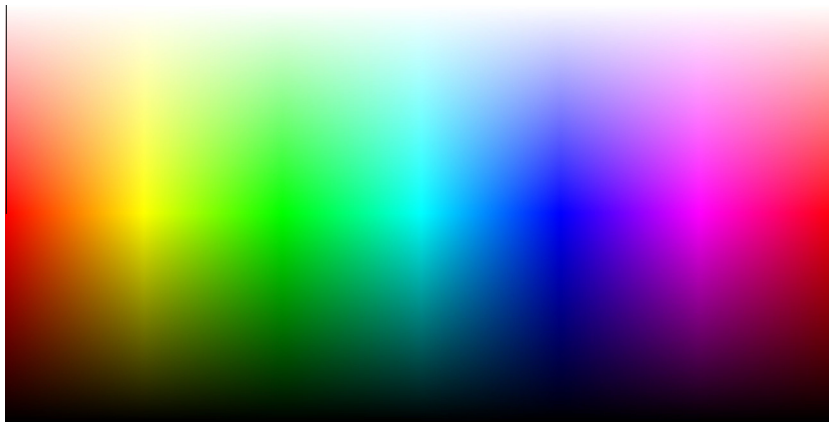


Fig. 3. Color palette generated by using integer-based HSV. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Table 3

Performance measurement results.

Configuration	Real-valued algorithm [ms]	Integer-based algorithm [ms]	Ratio
Debug	979	609	1.61
Release	492	249	1.98

The performance test was also based on the RGB color cube. For each color, two operations were performed: forward and backward conversion. The same was done for both the classical and the proposed algorithms under the same conditions. At least 1000 color cube iterations were performed and the best time for each algorithm was recorded. The aim was to measure the ratio of execution time for the real-valued algorithm to the integer-based algorithm. Such ratio represents approximate speed difference between the algorithms. The performance test results are shown in Table 3.

The proposed integer-based algorithm outperformed the classical real-valued one in both cases. The average ratio for the release version is 1.92. The source code of all the tests can be found in the author's repository [21]. The code is published under open source license.

The speed characteristics of the proposed algorithm were compared with similar integer-based algorithm. The most relevant and state-of-the-art algorithm found by the authors is the one, presented by Liu et al. [18]. Both algorithms share similar ideas, but they differ in realization.

Liu et al. [18] designed RGB to HSV conversion for fixed-point digital signal processors with the help of lookup tables (LUT). This eliminates the need of division operations but increases the memory usage. Note, that the use of LUT would increase the speed of the proposed algorithm as well.

The algorithm described by Liu et al. [18] is not accurate. As stated in the article, the maximum error can be equal to 3. Also, it does not include the backward conversion (from HSV to RGB). However, it is interesting to do a timing comparison of both integer-based algorithms for RGB to HSV conversion.

The test was based on the color cube method and executed on the same platform. The Liu algorithm slightly outperformed the presented algorithm. 335 [ms] against 336 [ms] for debug and 106 [ms] against 147 [ms] for release versions in average. This shows the importance of the LUT.

8. Conclusions

While developing the proposed algorithm, the main requirement was to provide a lossless conversion from the 24-bit RGB to HSV without the use of floating-point variables. This significantly increases performance and portability. The known integer-based algorithms either introduce small errors or use lookup tables in order to compensate it. After analysis of the existing methods, the authors accepted the challenge to develop a better solution. The result is presented in this paper.

The proposed algorithm is absolutely accurate. All the measured errors during the experiments were equal to 0. Lookup tables are not involved. There are no floating-point variables inside the conversion function, only integer and bit operations. As a result, about two times average performance gain, in comparison with the classical implementation, was achieved. The memory usage was minimized. The integer-based HSV color can be packed into 48 bits (6 bytes), which is less than 3 floats ($3 \cdot 32 = 96$ bits or 12 bytes).

More significant performance gain is expected on embedded system, especially where floating-point math is not implemented in hardware. In future it is planned to port the proposed algorithm into different architectures including DSP and FPGA.

Acknowledgment

The authors would like to acknowledge Michael Haldin (Metsähallitus Oy, Vaasa, Finland) for financial support, fruitful discussions and help.

References

- [1] Smith Alvy Ray. Color gamut transform pairs. *ACM siggraph computer graphics*, vol. 12. ACM; 1978. p. 12–9.
- [2] Ganesan P, Rajini V, Sathish BS, Shaik Khamar Basha. HSV color space based segmentation of region of interest in satellite images. In: 2014 International conference on control, instrumentation, communication and computational technologies (ICCICCT). IEEE; 2014. p. 101–5.
- [3] Liu Chunxue, Lu Xiaobo, Ji Saiping, Geng Wei. A fog level detection method based on image HSV color histogram. In: 2014 International conference on progress in informatics and computing (PIC). IEEE; 2014. p. 373–7.
- [4] Feng Liu, Xiaoyu Liu, Yi Chen. An efficient detection method for rare colored capsule based on RGB and HSV color space. In: 2014 IEEE international conference on granular computing (GrC). IEEE; 2014. p. 175–8.
- [5] Rahman Md Arifur, Purnama Edy, Purnomo Mauridhi Hery, et al. In: *Intelligent autonomous agents. 2014 IEEE international conference on networks and systems (INAGENTSYS)*. IEEE Press; 2014. p. 58–61.
- [6] Bhowmick Sourav, Talukdar Anjan Kumar, Sarma Kandarpa Kumar. Continuous hand gesture recognition for English alphabets. In: 2015 2nd International conference on signal processing and integrated networks (SPIN); February 2015. p. 443–6.
- [7] Chaudhary Ankit, Gupta Ankur. Automated switching system for skin pixel segmentation in varied lighting. In: 2012 19th International conference mechatronics and machine vision in practice (M2VIP). IEEE; 2012. p. 26–31.
- [8] Rashno Abdolreza, Sadri Saeed, SadeghianNejad Hossein. An efficient content-based image retrieval with ant colony optimization feature selection schema based on wavelet and color features. In: 2015 International symposium on artificial intelligence and signal processing (AISP). IEEE; 2015. p. 59–64.
- [9] Raja R, Roomi S, Dharmalakshmi D. Robust indoor/outdoor scene classification. In: 2015 Eighth international conference on advances in pattern recognition (ICAPR). IEEE; 2015. p. 1–5.
- [10] Archana T, Venugopal T, Praneeth Kumar M. Multiple face detection in color images. In: 2015 International conference on signal processing and communication engineering systems (SPACES). IEEE; 2015. p. 82–5.
- [11] Wang Wenyi, Zhao Jiying. Robust image chroma-keying: a quadmap approach based on global sampling and local affinity. *IEEE Trans Broadcast* 2015; PP(99). 1–1.
- [12] Chernov Vladimir, Koljonen Janne Tapio, Alander Jarmo, Bochko Vladimir. Video enhancement methods for monitoring submersible biological resources. In: *Agricontrol*, vol. 4; 2013. p. 293–8.
- [13] Guo Jun-Kai, Sung Chia-Chi, Chang Heng-Hua. Improving visibility and fidelity of underwater images using an adaptive restoration algorithm. In: *OCEANS 2014-TAIPEI*. IEEE; 2014. p. 1–6.
- [14] Paschos George. Perceptually uniform color spaces for color texture analysis: an empirical evaluation. *IEEE Trans Image Process* 2001;10(6):932–7.
- [15] Mattis Peter, Kimball Spencer. Source code file gimpcolospace.c from GIMP 2.8; 1995–1997. <<https://git.gnome.org/browse/gimp/tree/libgimpcolor/gimpcolospace.c?h=gimp-2-8&id=c086e98029411c8c4c78e166e697aad8a5686e92>>. Commit: c086e98029411c8c4c78e166e697aad8a5686e92.
- [16] Kabi Bibek, Sahoo Subhasmita, Samantaray Amiya Kumar, Routray Aurobinda. Floating to fixed-point translation with its application to speech-based emotion recognition. In: 2014 Fourth international conference of emerging applications of information technology (EAIT). IEEE; 2014. p. 21–6.
- [17] Dobashi Toshiyuki, Murofushi Tatsuya, Iwahashi Masahiro, Kiya Hitoshi. A fixed-point tone mapping operation for HDR images in the RGBE format. In: *Signal and information processing association annual summit and conference (APSIPA). 2013 Asia-Pacific*. IEEE; 2013. p. 1–4.
- [18] Liu Zhao-Guang, Du Sheng-Yong, Yang Yang, Ji Xiu-Hua. A fast algorithm for color space conversion and rounding error analysis based on fixed-point digital signal processors. *Comput Electr Eng* 2013.
- [19] Poynton Charles. A guided tour of color space. In: *SMPTE advanced television and electronic imaging conference*; 1995. p. 167–80.
- [20] Sarifuddin M, Missaoui Rokia. A new perceptually uniform color space with associated color similarity measure for content-based image and video retrieval. In: *Proc. of ACM SIGIR 2005 workshop on multimedia information retrieval (MMIR 2005)*; 2005. p. 1–8.
- [21] Chernov Vladimir. Proposed algorithm source code; 2014. <<https://bitbucket.org/chernov/colormath>>.
- [22] Fog Agner. Optimizing software in C++: an optimization guide for Windows. Linux and Mac platforms; 2014.

Vladimir Chernov received the B.S. and M.S. degrees in computer engineering from St.Petersburg Polytechnic University, Russia. In 2012 he started his Ph.D. studies in automation at University of Vaasa, Finland. His current research interest is on underwater video processing methods.

Jarmo T. Alander received the M.S. degree in physics from Helsinki University, Finland, the M.S. degree in computer science, the Ph.D. degree in computer science and engineering from Helsinki University of Technology. He has been a full Professor of production automation at the University of Vaasa, Finland since 1993. His main research interests include optimization using genetic algorithms, signal processing, automation, and computing in general.

Vladimir Bochko has the M.S. in telecommunications from St.Petersburg State University of Telecommunications, Russia, and the Ph.D. in technical science from St.Petersburg State Electrotechnical University. He is currently an Associate Professor at University of Vaasa, Finland. His research interests include spectral and color image analysis.